

```
print("Computer Programming 2026\n")  
print("Department of Geography\n")  
print("26 March 2026\n")  
  
print("Week 5")
```

```

def fact(val):
    factorial = 1
    for i in range(val, 1, -1):
        factorial = factorial * i
    return factorial

while True:
    K = int(input("\nInsert the value of K: "))
    N = int(input("Insert the value of N: "))
    if K >= 0 and N >= K:
        break

FactK = fact(K)
FactN = fact(N)
FactNK = fact(N-K)
BinCoef = FactN / (FactK * FactNK)
print(f"\nThe Binomial Coefficient with N = {N} and K = {K} is {int(BinCoef)}")

```

RECURSION

Recursion

Definition:

- From latin **re-currere**
 - recur, repeat the same action
- In computer science: related to functions that **call themselves**

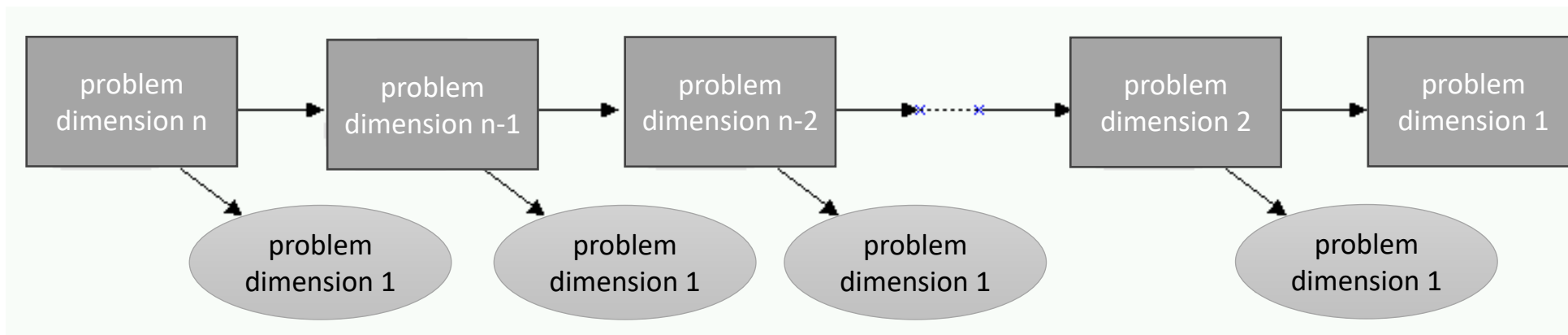
The purpose is to solve a problem referring to the same problem at a reduced scale.

The **ending condition** is satisfied when one or more **simple cases with immediate solution** are identified.

Purpose of recursive programming

The structure of a recursive algorithm is the following:

```
if (it is the simple case) :  
    solve it  
else:  
    use the recursion on reduced data
```



Recursion: What is it?

- Indirect recursion:
 - a subprogram P calls a subprogram Q
 - Q then calls a third subprogram R, ...
 - R calls again P
- Direct recursion
 - A subprogram P calls itself during its execution

The factorial

Given n , positive integer, we define the n factorial (represented as $n!$) the product of the first n positive integer numbers that are lower or equal to that number.

$$n! := \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

Iterative factorial

```
def fact(val):  
  
    factorial = 1  
  
    for i in range(val, 1, -1):  
        factorial = factorial * i  
  
    return factorial
```


Recursive definition of the factorial

$$1) \ n! = 1 \quad \text{if } n=0$$

$$2) \ n! = n * (n-1)! \quad \text{if } n>0$$

- It reduces the calculation to a simpler calculation
- It makes sense because it is always based on the factorial of the smaller number
- It makes sense because we get to some point where it is not necessary anymore to reuse the def. 2) and we instead use the def. 1)
- 1) is the basic step, 2) is the recursion step

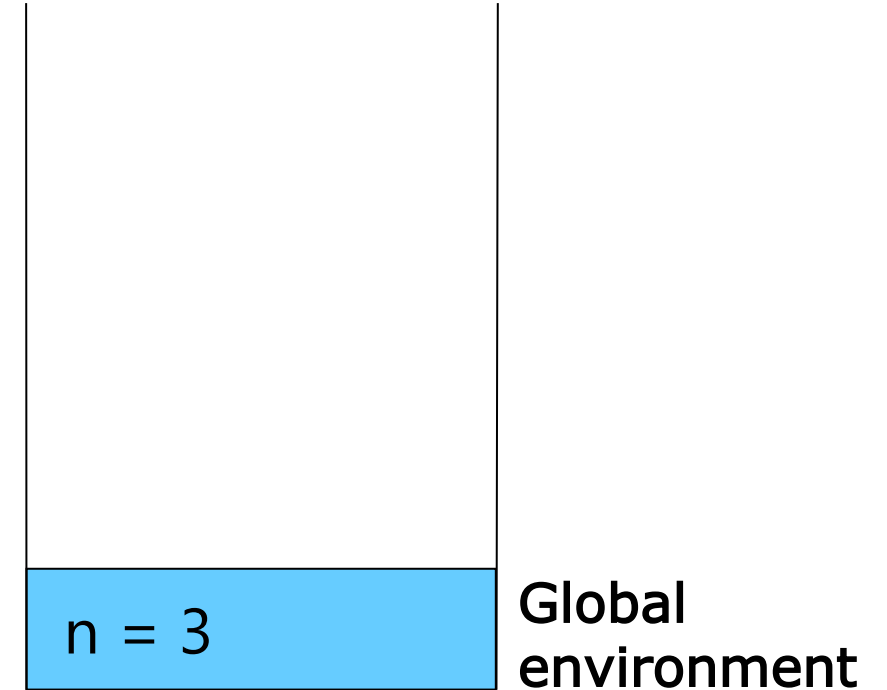
Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```



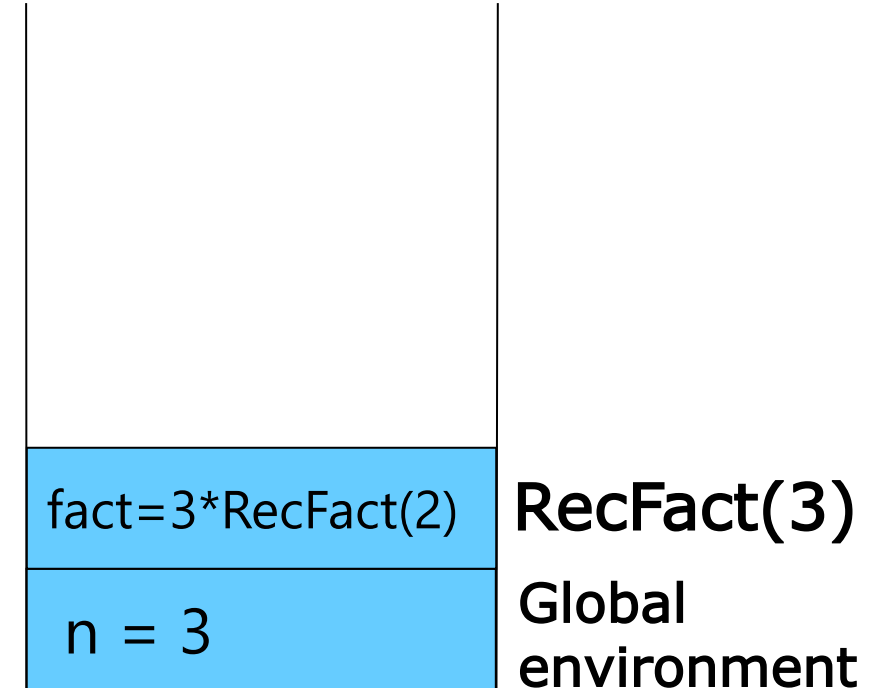
Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```



Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact=2*RecFact(1)	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact=1*RecFact(0)	RecFact(1)
fact=2*RecFact(1)	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact = 1	RecFact(0)
fact=1*RecFact(0)	RecFact(1)
fact=2*RecFact(1)	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact = 1	RecFact(0)
fact=1*RecFact(0)	RecFact(1)
fact=2*RecFact(1)	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact = 1	RecFact(0)
fact = 1 * 1	RecFact(1)
fact=2*RecFact(1)	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
  
    return factorial
```

fact = 1	RecFact(0)
fact = 1	RecFact(1)
fact = 2 * 1	RecFact(2)
fact=3*RecFact(2)	RecFact(3)
n = 3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
    return factorial
```

fact =	1	RecFact(0)
fact =	1	RecFact(1)
fact =	2	RecFact(2)
fact =	3 * 2	RecFact(3)
n =	3	Global environment

Recursive factorial

Calculate the factorial in the recursive way:

Fact(0) = 1

Fact(n) = n*Fact(n-1)

```
def RecFact(n):  
    if n == 0:  
        factorial = 1  
    else:  
        factorial = n * RecFact(n-1)  
  
    return factorial
```

fact =	1	RecFact(0)
fact =	1	RecFact(1)
fact =	2	RecFact(2)
fact =	6	RecFact(3)
6		Global environment

Other examples of recursive functions

- Multiplication
- Fibonacci numbers (population dynamics)
- Tower of Brahma

Multiplication

Design a recursive process for calculating the product of two integers:

Note: $A*1=A$; $A*B = A + A*(B-1)$

```
def RecMul(a, b):  
    if b == 1:  
        result = a  
    else:  
        result = a + RecMul(a, b-1)  
    return result
```

Fibonacci

Leonardo Fibonacci

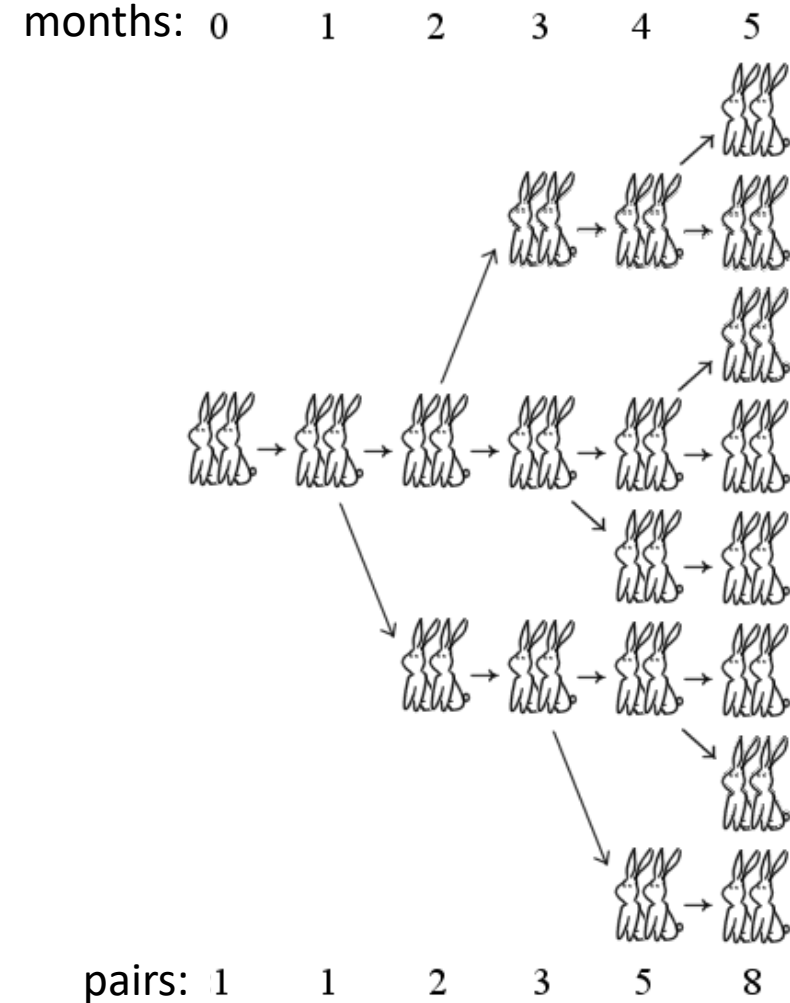
- Italian mathematician
- After several journeys, he assimilated the mathematical knowledge of the Arab world and introduced its number system and calculation rules into the western scientific environment
- In 1202, he published the *Liber abaci*



The “rabbit” problem

“A man has one pair of rabbits at a certain place entirely surrounded by a wall. We wish to know how many pairs will be bred from it in one year, if the nature of these rabbits is such that they breed every month one other pair and begin to breed in the second month after their birth.”

L. Fibonacci from Liber Abaci

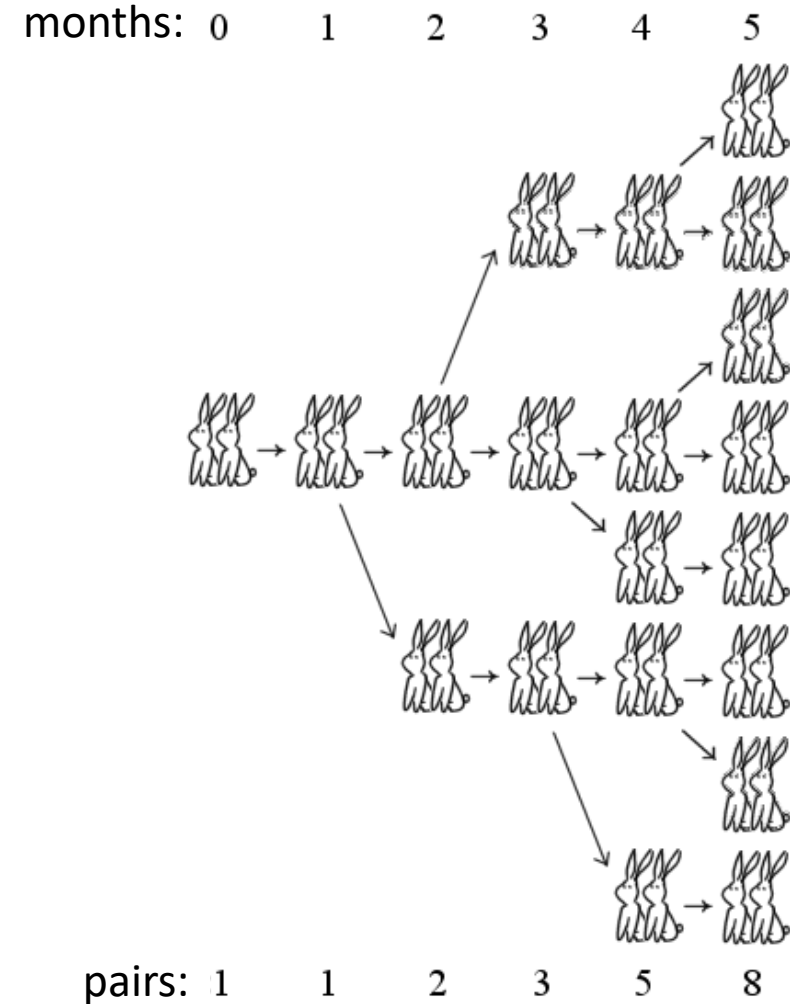


Fibonacci numbers

Underlying idea:

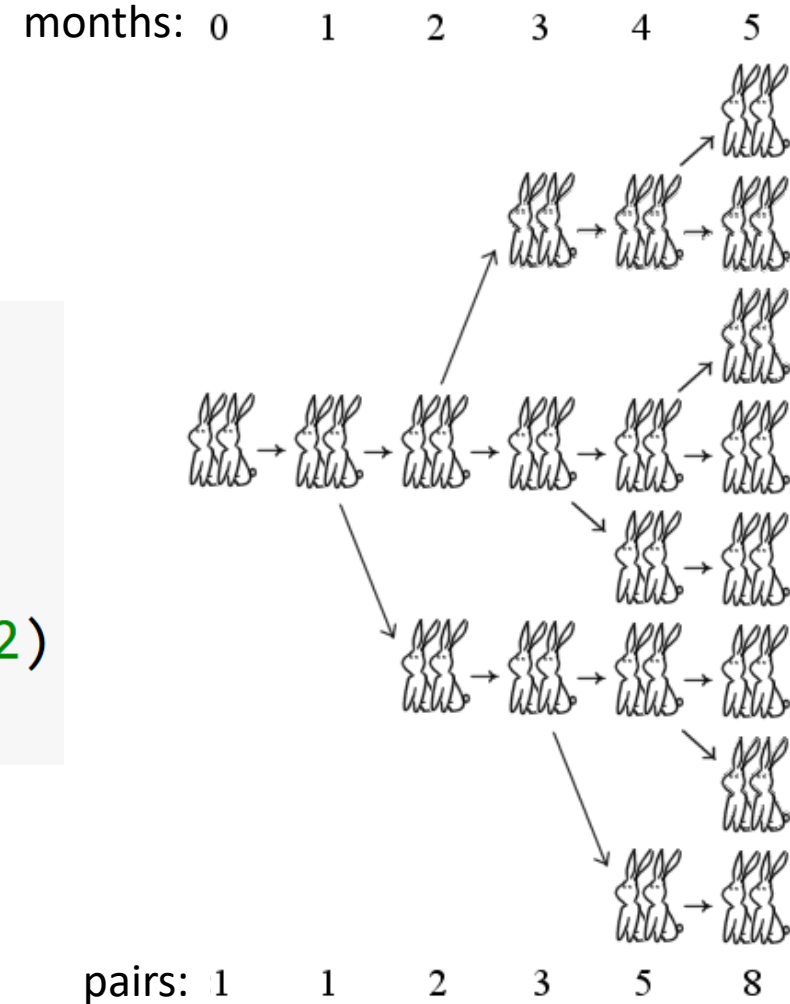
1) $\text{fib}(n)=1$ if $n=0$ or $n=1$

2) $\text{fib}(n)= \text{fib}(n-1) + \text{fib}(n-2)$ if $n>1$

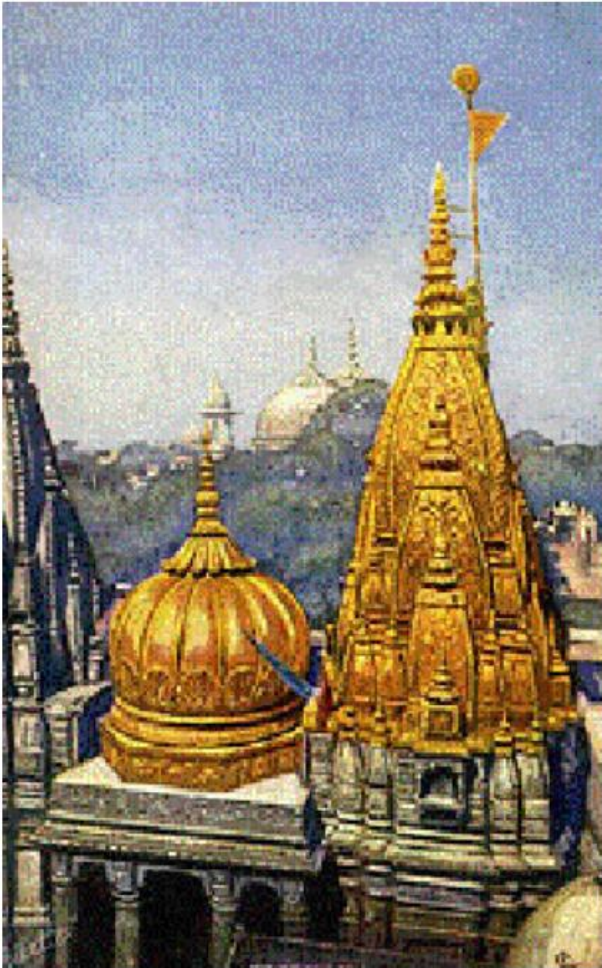


Fibonacci sequence

```
def RecFib(n):  
    if n == 0 or n == 1:  
        result = 1  
    else:  
        result = RecFib(n-1) + RecFib(n-2)  
    return result
```



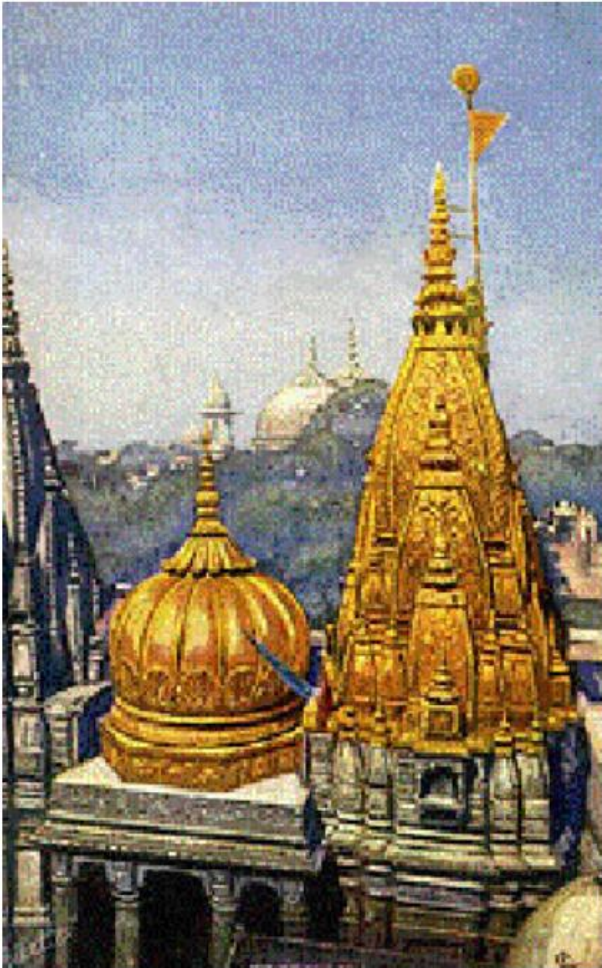
An interesting problem: the Tower of Brahma



Edouard Lucas, 1842 - 1891

- A legend says that when the world was created, Brahma brought, under the golden dome of Benares temple, at the center of the world, three small towers made of diamond and sixty-four disks made of gold.
- All disks were placed on the first tower in descending order, from the largest one on the bottom to the smallest one on the top.

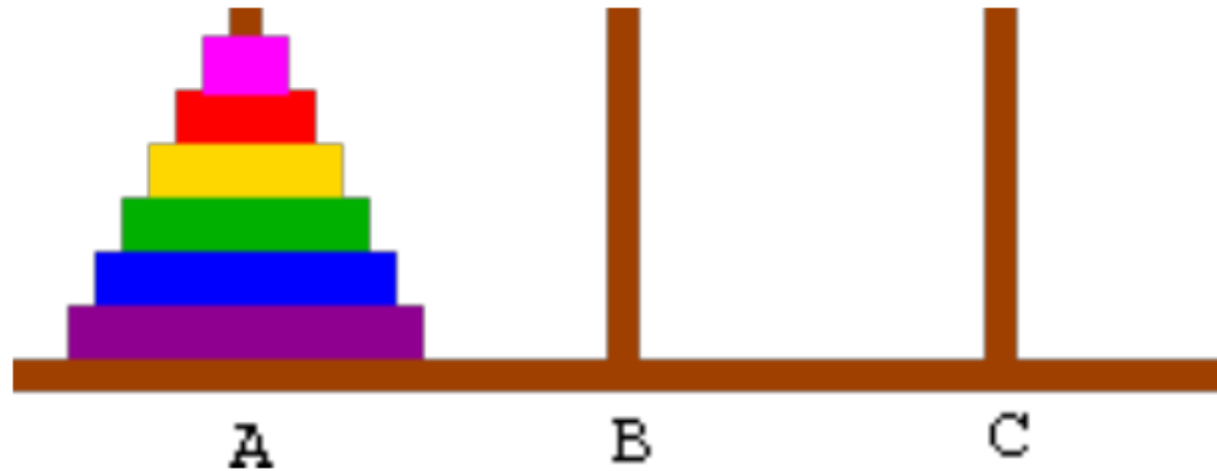
An interesting problem: the Tower of Brahma



Edouard Lucas, 1842 - 1891

- The priests of the Sacred Tower of Brahma are working day and night with the purpose of moving the disks from the first tower to the second tower.
- The priests are not allowed to disobey the strict rules set up by Brahma: 1) it is possible to move only one disk at a time; 2) a bigger disk can never be placed over a smaller disk.
- **When the priests finish their work with all disks sorted on the second tower, the temple will explode and the world will be destroyed.**

Tower of Brahma



Problem:

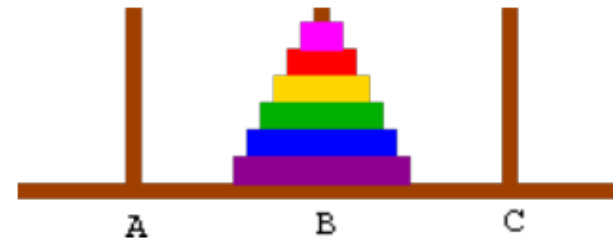
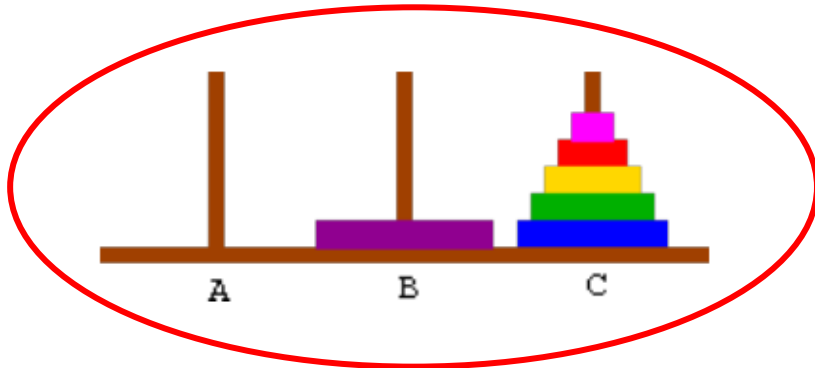
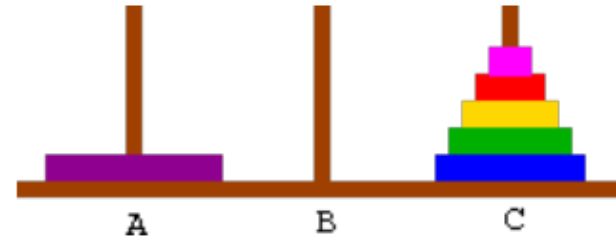
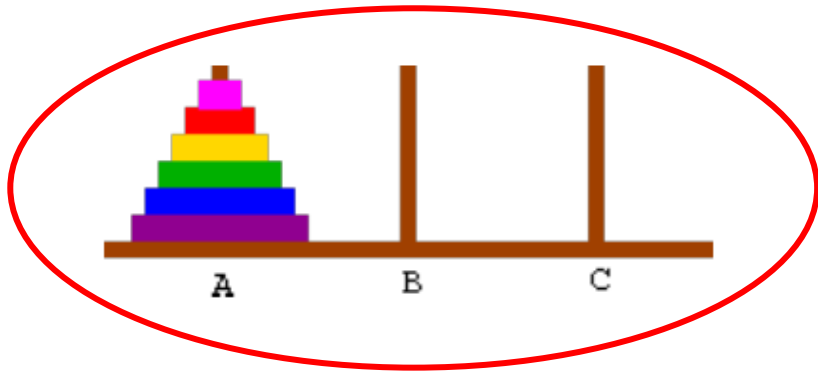
Move all disks from tower A to tower B (using tower C as an “intermediate support”) in such a way to sort the disks according to the same original order

Tower of Brahma (underlying idea)

I want to move n disks from the source tower to the destination tower, using the spare tower as a support

- First, I have to move $n - 1$ disks from the source to the spare tower, using the destination tower as a support
- Then, I move the only remaining disk from the source to the destination tower
- Finally, I move the $n - 1$ disks from the spare tower to the destination tower

Tower of Brahma (strategy)



The problem is reduced to the case of moving five disks from tower C to tower B

Tower of Brahma

We will write a recursive function whose input parameter is the number of disks we want to move

The function will also have three additional parameters indicating:

- which tower we want to start from (source),
- which tower we want to get to (destination),
- the other tower we can use as a temporary support (spare).

Tower of Brahma (recursion)

- When we move the $n - 1$ disks, the function calls itself (i.e., executes a recursive call), but simplifying the problem, as we need to move a smaller number of disks.
- With the recursion, the problem gradually reduces its complexity until reaching the trivial solution (i.e., until only one disk is left, which is simply moved to the destination tower).

Note: The basic case in the recursion program refers to “only one disk”, which can be directly moved because it does not have any other disks on top.

Tower of Brahma (code)

```
def brahma(n, source, dest, spare):  
    if n == 1:  
        print("I move from", source, "to", dest)  
    else:  
        brahma(n-1, source, spare, dest)  
        brahma(1, source, dest, spare)  
        brahma(n-1, spare, dest, source)  
    return
```


Exercise

- Given 3 integers inserted by the user, display such 3 integers in the **reversed order** of insertion

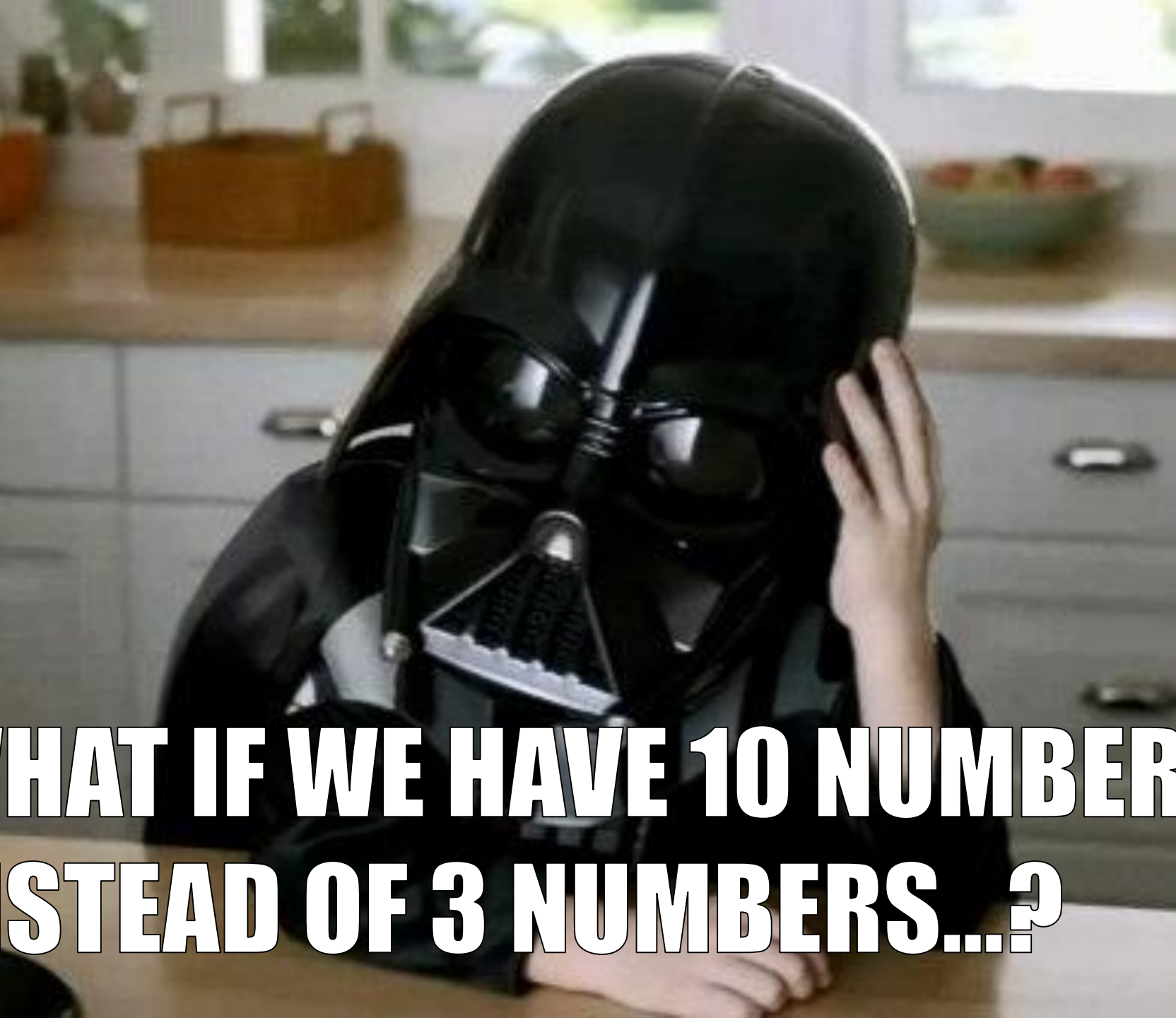
```
first = int(input("Insert the first number: "))
second = int(input("Insert the second number: "))
third = int(input("Insert the third number: "))

print("\nThe first inserted number is:", first)
print("The second inserted number is:", second)
print("The third inserted number is:", third)

print("\nThe third inserted number is:", third)
print("The second inserted number is:", second)
print("The first inserted number is:", first)
```

In memory:

- first
- second
- third

A person wearing a Darth Vader helmet is sitting at a kitchen table, resting their head on their hand in a distressed or thoughtful pose. The background shows a kitchen with a countertop, a basket, and a bowl of fruit.

**WHAT IF WE HAVE 10 NUMBERS
INSTEAD OF 3 NUMBERS...?**

List

List

- Group of consecutive cells
 - Representation of variable groups
 - With the same name (but not necessarily the same type)
- To refer to an ***element***, we must specify:
 - The name of the list
 - The ***position*** of the element (index)
- Syntax: *list_name*[*element_position*]
 - The first element has index **0**
 - The n-th element of the list **v** is **v[n-1]**

list name

v[0]	-45
v[1]	6
v[2]	0
v[3]	72
v[4]	1543
v[5]	-89
v[6]	0
v[7]	62
v[8]	-3
v[9]	1
v[10]	6453
v[11]	78

element position

List

- The elements of a list are simple variables:

```
mylist[0] = 3
```

```
print(mylist[0])
```

```
> 3
```

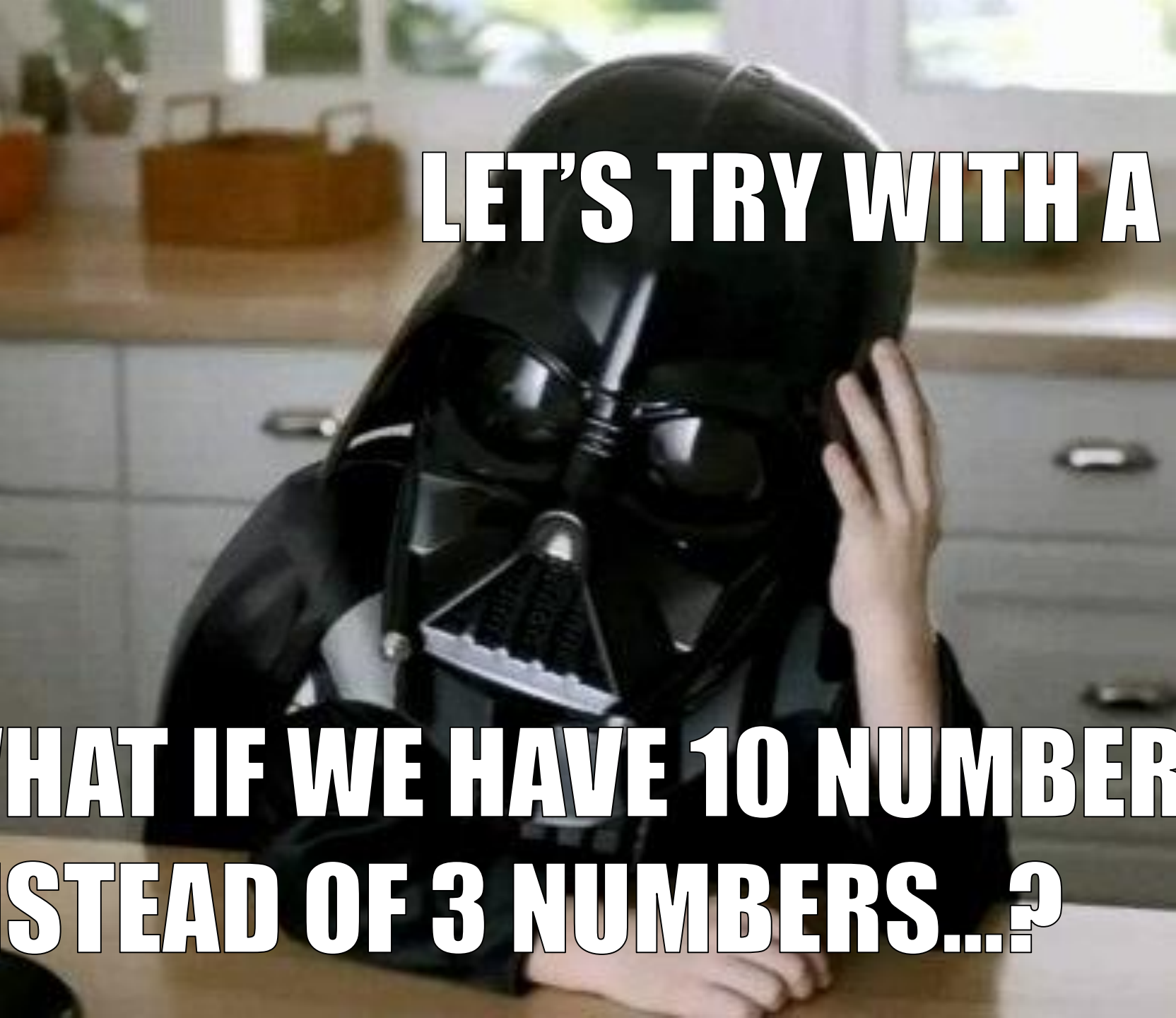
```
mylist[1] = input("...: ")
```

```
> 17          mylist[1] assumes value 17
```

- You can also use **expressions** as indices:

```
mylist[5-2] is equivalent to mylist[3]
```

```
if x = 3, then mylist[x] is equivalent to mylist[3]
```

A person in a Darth Vader costume is sitting at a kitchen counter, looking down with their hand to their forehead in a frustrated or overwhelmed expression. The background shows a typical kitchen with cabinets and a window.

LET'S TRY WITH A LIST!

**WHAT IF WE HAVE 10 NUMBERS
INSTEAD OF 3 NUMBERS...?**

Exercise

- Given 10 integers inserted by the user, display such 10 integers in the **reversed order** of insertion

```
data = []
```

How to *save* the numbers... the insertion

- Problem 1: I have to save data in my list

```
data = []
```

- Problem 2: I have to define a loop that allows me to scan 10 elements

How to *save* the numbers... the insertion

- Problem 1: I have to save data in my list

```
data = []
```

```
data.append(input("...: "))
```

How to *save* the numbers... the insertion

- Problem 2: I have to define a loop that allows me to scan 10 elements

```
for i in range(10) :
```

How to *save* the numbers... the insertion

- Problem 1 + Problem 2

```
for i in range(10):  
    data.append(input("...: "))
```

Exercise

- Given 10 integers inserted by the user, display such 10 integers in the **reversed order** of insertion

```
data = []  
  
for i in range(10):  
    data.append(int(input(f"Insert the number (choice {i+1}): ")))
```

Let's print out the numbers...

```
for i in range(10):  
    print("The number is: ", data[position])
```

Wrong!!!
1) I always read from the same cell
2) What is the value of «position»???

Let's print out the numbers...

```
position = 0
for i in range(10):
    print("The number is: ", data[position])
    position += 1
```

How do «i» and «position» grow?

Let's print out the numbers...

```
for i in range(10):  
    print("The number is: ", data[i])
```

Exercise

- Given 10 integers inserted by the user, display such 10 integers in the **reversed order** of insertion

```
data = []

for i in range(10):
    data.append(int(input(f"Insert the number (choice {i+1}): ")))

print("\n")
for i in range(10):
    print(f"The number (choice {i+1}) is: {data[i]}")
```

Exercise

```
data = []

for i in range(10):
    data.append(int(input(f"Insert the number (choice {i+1}): ")))

print("\n")
for i in range(10):
    print(f"The number (choice {i+1}) is: {data[i]}")

print("\n")
for i in range(10, 0, -1):
    print(f"The number (choice {i}) is: {data[i-1]}")
```

Exercise

What if now I want 100 numbers?

```
data = []

for i in range(10):
    data.append(int(input(f"Insert the number (choice {i+1}): ")))

print("\n")
for i in range(10):
    print(f"The number (choice {i+1}) is: {data[i]}")

print("\n")
for i in range(10, 0, -1):
    print(f"The number (choice {i}) is: {data[i-1]}")
```

Let's “generalize” with a variable

- At the beginning of the program:
LENGTH = 100
- In this way we can adapt the length (number of loops) to potential future modifications without rewriting the constant 100 in many parts of the program
 - Before execution, **LENGTH** is replaced with **100**
- However, please note that the length of a list does not need to be specified in advance, but it is allowed to change during execution

Exercise

```
data = []
DIM = 100

for i in range(DIM):
    data.append(int(input(f"Insert the number (choice {i+1}): ")))

print("\n")
for i in range(DIM):
    print(f"The number (choice {i+1}) is: {data[i]}")

print("\n")
for i in range(DIM, 0, -1):
    print(f"The number (choice {i}) is: {data[i-1]}")
```

Exercise

What if now I want 17 numbers?

```
data = []  
DIM = 100  
  
for i in range(DIM):  
    data.append(int(input(f"Insert the number (choice {i+1}): ")))  
  
print("\n")  
for i in range(DIM):  
    print(f"The number (choice {i+1}) is: {data[i]}")  
  
print("\n")  
for i in range(DIM, 0, -1):  
    print(f"The number (choice {i}) is: {data[i-1]}")
```

A short summary...

- List: collection of data
 - Its elements may be of same or different types
 - Declare an empty list before its use

```
data = []
```

- How to access data in a list?

- Example of *writing*

```
for i in range(10):  
    data.append(int(input(f"Insert the number (choice {i+1}): ")))
```

- Example of *reading*

```
for i in range(10):  
    print(f"The number (choice {i+1}) is: {data[i]}")
```


See you
next week!